

Architecture Decisions & Assumptions

Project Overview

Production-quality REST API for restaurant ordering system with focus on security, performance, and maintainability.

Core Assumptions

1. User Roles & Permissions

Roles Implemented:

- CUSTOMER: Can place orders, view own orders, browse menus
- RESTAURANT_OWNER: Can manage own restaurant, menu, categories, dishes, view restaurant orders
- ADMIN: Full system access, manage all restaurants

Authorization Logic:

- Customers can only view/manage their own orders
- Restaurant owners can only manage their assigned restaurant
- Orders are restaurant-specific (single restaurant per order)
- Public endpoints: menu browsing, restaurant listing (read-only)

2. Data Model Relationships

- **User ↔ Restaurant:** One owner can have multiple restaurants (future-proof)
- **Restaurant ↔ Menu:** One-to-one (each restaurant has one menu)
- **Menu ↔ Categories:** One-to-many
- **Category ↔ Dishes:** One-to-many
- **Restaurant ↔ Orders:** One-to-many
- **Order ↔ OrderItems:** One-to-many
- **User ↔ Orders:** One-to-many (customer relationship)

3. Order Management

Order Status Flow:



PENDING → CONFIRMED → PREPARING → READY → COMPLETED
↘ CANCELLED (can happen at any stage before READY)

Status Update Permissions:

- PENDING → CONFIRMED: Restaurant owner
- CONFIRMED → PREPARING: Restaurant owner
- PREPARING → READY: Restaurant owner
- READY → COMPLETED: Restaurant owner or system (after delivery)

- * → CANCELLED: Customer (only PENDING), Restaurant owner (any stage)

4. Restaurant Schema

Fields Included:

- Basic: name, description, address, phone, email
- Operations: openingHours (JSON), cuisineType, priceRange
- Media: imageUrl, logoUrl
- Status: isActive (soft enable/disable)
- Ownership: ownerId (FK to Users)
- Audit: createdAt, updatedAt

5. Menu & Ratings

- **Ratings:** Dish-level ratings (1-5 stars)
- **Categories:** Can be marked unavailable (seasonal menus)
- **Dishes:** Individual availability flag
- **includeUnavailable:** When true, returns all items regardless of availability status

Sorting Support:

- Dishes: price, name, averageRating, createdAt
- Categories: displayOrder, name

6. Performance Considerations

Indexes Added:

- Orders: (restaurantId, status, createdAt) - composite for reporting queries
- Orders: (userId, createdAt) - user order history
- Orders: (createdAt) - time-based queries
- Dishes: (categoryId, isAvailable)
- OrderItems: (orderId)

Optimizations:

- Pagination on all list endpoints (default: 20 items)
- Eager loading for related data to avoid N+1 queries
- Database connection pooling
- Query result caching for menu data (Redis bonus)

Security Decisions

Authentication

- **JWT Access Token:** 15 minutes expiry
- **JWT Refresh Token:** 7 days expiry, rotation on use
- **Password Hashing:** bcrypt with 12 rounds
- **Rate Limiting:**
 - Auth endpoints: 5 requests/15 minutes per IP
 - API endpoints: 100 requests/15 minutes per IP
- **Brute Force Protection:** Account lockout after 5 failed login attempts (15 min cooldown)

Input Validation

- Zod schemas for all request payloads
- Email normalization (lowercase, trim)
- SQL injection prevention via Sequelize parameterized queries
- XSS prevention via input sanitization

Authorization

- JWT middleware on protected routes
- Role-based access control (RBAC)
- Resource ownership verification for modify operations

Technical Stack Rationale

Technology	Justification
TypeScript	Type safety, better IDE support, catches errors at compile time
Express.js	Mature, lightweight, extensive middleware ecosystem
Sequelize	Full-featured ORM, TypeScript support, migration system
MySQL	Relational data with complex queries, ACID compliance needed
Jest/Supertest	Industry standard, easy mocking, snapshot testing
Zod	Runtime validation, TypeScript inference, composable schemas
bcrypt	Battle-tested, appropriate cost factor configuration

API Design Principles

- RESTful conventions
- Consistent error response format
- Pagination via query params: `?page=1&limit=20`
- Filtering via query params: `?status=PENDING&from=2025-01-01`
- Sorting via query param: `?sortBy=price&order=asc`
- Versioning ready: `/api/v1/...`

Error Handling Strategy

- Custom error classes (ValidationError, AuthError, NotFoundError)
- Global error handler middleware
- Consistent JSON error responses:



json

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [...]
  }
}
```

Testing Strategy

- Unit tests: Services, utilities, validators
- Integration tests: API endpoints with test database
- Test coverage target: 70%+
- Test data: Factories for consistent test fixtures

Deployment Considerations

- Environment-based configuration
- Database migrations for version control
- Seed scripts for development/staging
- Health check endpoint: /api/health
- Graceful shutdown handling

AI Tools Used

- **Windsurf/Cursor:** Code generation, boilerplate scaffolding
- **Claude (Anthropic):** Architecture review, query optimization, documentation
- **GitHub Copilot:** Code completion, test generation

Future Enhancements (Out of Scope)

- Payment integration
- Real-time order tracking
- Multi-language support
- Advanced analytics dashboard
- Mobile push notifications